



UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Implementación de un Sistema de Planificación de Rutas Óptimas Multimodales en el
Transporte Público de Santiago

PROPUESTA DE TEMA DE MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL EN COMPUTACIÓN

Javier Andrés Facondi Moran

MODALIDAD:
Memoria

PROFESOR GUÍA:
Eduardo Graells G.

SANTIAGO DE CHILE
2025

1. Introducción

En Santiago, combinar el Metro con un bus del sistema RED no es una operación matemática abstracta; implica caminar por veredas reales, esperar en paraderos específicos y lidiar con tiempos de transbordo que el sistema actual a menudo ignora. Con una población que supera los 7 millones de habitantes, la capacidad de planificar viajes eficientes no es solo una necesidad logística, sino un factor determinante en la calidad de vida de las personas.

Para que un sistema de planificación sea útil, debe ser capaz de replicar esta realidad del usuario, quien raramente realiza trayectos directos de puerta a puerta en un solo vehículo. La movilidad real es inherentemente multimodal: implica caminar hacia una estación, combinar diferentes líneas de metro o conectar un servicio de bus con un tren.

1.1. Contexto del Proyecto

Esta memoria continúa el trabajo del proyecto *Ayatori* [1], desarrollado en el Departamento de Ciencias de la Computación de la Universidad de Chile. El objetivo original fue crear un módulo en Python capaz de procesar la inmensa cantidad de datos de transporte usando estándares abiertos como GTFS y OpenStreetMap.

Si bien Ayatori estableció una base técnica sólida, su implementación preliminar del algoritmo *Connection Scan Algorithm* (CSA) en variante "Lite"[2] dejó pendiente la complejidad real de los transbordos, funcionando más como una prueba de concepto que como un planificador viable.

1.2. Definición del Problema

A pesar de los avances, la implementación actual posee una limitación crítica: la incapacidad de modelar transbordos. La versión simplificada del algoritmo CSA asume erróneamente que los viajes ocurren en un único vehículo [1].

Esta omisión reduce drásticamente la utilidad del sistema en una ciudad interconectada como Santiago. Actualmente, el sistema no puede responder a consultas cotidianas como “¿Cómo llego desde Beauchef 851 hasta la Biblioteca Municipal de Pudahuel?”, ya que dicha ruta requiere inevitablemente combinar caminata, Metro y Bus.

El desafío no es solo lógico (conectar grafos), sino espacial. Un transbordo real implica un segmento de caminata que debe ser optimizado. Ignorar este factor genera rutas imposibles de ejecutar en la realidad física [7].

1.3. Propuesta de Solución

Esta propuesta busca superar la barrera de los transbordos mediante la implementación completa del *Connection Scan Algorithm* (CSA). A diferencia de algoritmos tradicionales como Dijkstra, CSA optimiza rutas basándose en horarios y conexiones temporales de manera eficiente.

El objetivo central es dotar al módulo Ayatori de una lógica de enrutamiento robusta que

integre la **multimodalidad** para una conexión fluida entre servicios, pero con un enfoque crítico en los **transbordos realistas**. No basta con conectar dos líneas; es necesario calcular los tiempos de espera y penalizar los cambios de servicio. Asimismo, modelaremos explícitamente la **caminata**, tratando los segmentos a pie no como distancias euclidianas, sino como rutas físicas necesarias para conectar el origen, los puntos de transbordo y el destino final.

Al completar este desarrollo, el módulo dejará de ser un prototipo para convertirse en una herramienta funcional, permitiendo que planificadores utilicen datos concretos para evaluar la eficiencia de la red actual.

2. Estado del Arte

La planificación eficiente de rutas en sistemas de transporte público multimodal constituye un desafío fundamental en la ingeniería de transporte y la algoritmia moderna. Este problema, técnicamente denominado *Multimodal Shortest Path Problem* (MSPP), difiere sustancialmente de la navegación vehicular tradicional debido a la naturaleza dependiente del tiempo de los servicios de transporte y la complejidad de integrar múltiples modos de movilidad (caminata, bus, metro).

Esta sección presenta una revisión exhaustiva de los estándares de datos vigentes, la evolución teórica de los algoritmos de enrutamiento desde los enfoques basados en grafos hasta el escaneo de conexiones y analiza en detalle el estado actual del desarrollo tecnológico sobre el cual se fundamenta esta memoria.

2.1. Estándares de Datos para la Movilidad Urbana

La digitalización del transporte público ha convergido globalmente en dos estándares abiertos que permiten desacoplar la generación de datos del desarrollo de aplicaciones. La correcta integración de estos estándares es la base de cualquier sistema de enrutamiento moderno.

2.1.1. Infraestructura y Red Peatonal: OpenStreetMap

La base espacial del enrutamiento multimodal recae en la representación de la red de caminata. OpenStreetMap (OSM) proporciona una base de datos geoespacial colaborativa que modela la ciudad mediante tres primitivas topológicas: nodos (puntos georreferenciados), vías (listas ordenadas de nodos que representan calles o senderos) y relaciones.

En el contexto de la movilidad, OSM es crítico para resolver el problema de la “última milla” y los segmentos de transbordo [6]. A diferencia de los grafos viales simplificados, OSM permite calcular distancias geodésicas precisas y considerar barreras físicas, lo cual es esencial para determinar si un transbordo entre dos paraderos es humanamente viable dentro de un tiempo razonable.

2.1.2. Programación de Servicios: GTFS

La especificación GTFS (*General Transit Feed Specification*) es el estándar *de facto* para la publicación de itinerarios. Un *feed* GTFS modela el sistema mediante archivos relacionales CSV, siendo los más críticos para el enrutamiento:

- `stops.txt`: Define la ubicación georreferenciada de paradas y estaciones.
- `routes.txt`: Define los servicios lógicos (ej. recorrido 506).
- `stop_times.txt`: Contiene la matriz de tiempos de llegada y salida para cada viaje en cada parada.
- `calendar.txt`: Establece las reglas de operación (días hábiles, fines de semana).

Si bien GTFS permite una representación detallada, su naturaleza estática presenta limitaciones algorítmicas. Ortiz-Rosales et al. advierten que los sistemas basados puramente en GTFS estático asumen una operación ideal, ignorando la estocasticidad propia del tráfico

urbano, lo que puede resultar en rutas teóricamente óptimas pero inviables en la práctica ante retrasos o congestión [8].

2.2. Evolución de Algoritmos de Enrutamiento

Históricamente, el problema de la ruta más corta en redes de transporte público se ha abordado adaptando algoritmos clásicos de teoría de grafos. Sin embargo, el tamaño y la dinámica temporal de estas redes han impulsado el desarrollo de técnicas especializadas.

2.2.1. Enfoques Basados en Grafos (*Graph-Based*)

La literatura clásica, revisada por Müller y Zündorf, identifica dos modelos principales para representar el tiempo en grafos [4]:

1. **Time-Expanded Graphs (TEG):** Este modelo desenrolla el tiempo en el espacio. Cada evento de salida o llegada se convierte en un nodo único. Si bien permite aplicar el algoritmo de Dijkstra directamente, el tamaño del grafo crece linealmente con el número de eventos en el horario, resultando en millones de nodos para una red como la de Santiago, lo que lo hace ineficiente en memoria.
2. **Time-Dependent Graphs (TDG):** Mantiene la topología física de la red, pero las aristas poseen pesos dinámicos $w(t)$ que dependen del tiempo de llegada al nodo de origen. Aunque es más compacto que TEG, la evaluación de las funciones de peso durante la exploración del grafo añade una sobrecarga computacional significativa.

Estudios aplicados, como el de Hertell y Korhonen, han demostrado que algoritmos como A^* (A-Star) sobre estos grafos pueden ser efectivos si se diseñan heurísticas admisibles robustas, pero su rendimiento se degrada rápidamente en redes densas y multimodales [3].

2.2.2. Enfoques de Escaneo de Conexiones (*CSA*)

Para superar las ineficiencias de los grafos, Dibbelt et al. introdujeron el *Connection Scan Algorithm* (CSA), cambiando el paradigma de “exploración de grafos” a “escaneo de arreglos” [2].

Matemáticamente, CSA no utiliza un grafo $G = (V, E)$. En su lugar, organiza el horario como un arreglo ordenado de conexiones elementales. Una conexión c se define como una 5-tupla:

$$c = (S_{dep}, S_{arr}, t_{dep}, t_{arr}, trip_id)$$

Donde un vehículo viaja de la parada S_{dep} a S_{arr} , saliendo en el tiempo t_{dep} y llegando en t_{arr} .

El algoritmo itera una única vez sobre este arreglo ordenado cronológicamente. Mantiene un arreglo $A[s]$ que almacena el tiempo de llegada más temprano a la parada s . Para cada conexión c , si el usuario puede llegar a S_{dep} antes de t_{dep} (es decir, $A[S_{dep}] \leq t_{dep}$), entonces actualiza $A[S_{arr}]$ con t_{arr} .

Esta estructura ofrece ventajas críticas para esta memoria:

- **Localidad de Memoria:** Al acceder al arreglo de forma lineal, se maximiza el uso de la caché del procesador.
- **Simplicidad:** Evita el uso de colas de prioridad complejas ($O(1)$ por conexión vs $O(\log n)$ en Dijkstra).
- **Flexibilidad:** Facilita la incorporación de criterios multicriterio, como la minimización de transbordos.

2.3. El Desafío de la Multimodalidad y los Transbordos

La integración de múltiples modos de transporte introduce la complejidad de los transbordos no triviales. En un sistema unimodal, un transbordo suele modelarse como un evento instantáneo en el mismo nodo. En sistemas multimodales (ej. bajar del Metro y caminar a un paradero de bus), el transbordo implica un desplazamiento físico.

Zografos y Androutsopoulos destacan que ignorar la penalización del transbordo tanto en tiempo como en esfuerzo físico genera soluciones que los usuarios rechazan [9]. Para modelar esto correctamente, es necesario calcular dinámicamente los segmentos de caminata.

La literatura sugiere el uso de la fórmula de Haversine para calcular la distancia de círculo máximo entre dos puntos de la esfera terrestre (paradas), permitiendo filtrar conexiones viables dentro de un radio de caminata aceptable [7]. Este cálculo debe integrarse en el bucle principal del algoritmo de enrutamiento para considerar el tiempo de caminata como parte del costo total del viaje [3].

2.4. Antecedentes del Proyecto: El Módulo Ayatori

Esta memoria se construye sobre la base del proyecto *Ayatori*, desarrollado por Leal Cerro en la Universidad de Chile. Este módulo fue concebido como una herramienta *open source* para facilitar la investigación en algoritmos de transporte utilizando datos de Santiago [1].

2.4.1. Arquitectura y Estructura de Datos

El sistema Ayatori implementa una arquitectura modular que separa el procesamiento cartográfico de la lógica de transporte:

- **Clase OSMGraph:** Encargada de la ingestión de datos de OpenStreetMap. Originalmente basada en la librería `graph-tool` para un alto rendimiento en C++, esta clase gestiona la red vial y peatonal.
- **Clase GTFSData:** Responsable del *parsing* y estructuración de los archivos GTFS. Convierte los archivos CSV crudos en estructuras de memoria optimizadas para consultas rápidas de horarios y secuencias de paradas.

2.4.2. Limitaciones Críticas y Deuda Técnica

A pesar de proporcionar una base sólida para la carga de datos, la versión actual de Ayatori presenta limitaciones que impiden su uso como un planificador de viajes funcional. Estas brechas, identificadas en la documentación del proyecto y corroboradas en el análisis preliminar, son los objetivos directos de esta memoria:

1. **Algoritmo CSA Incompleto (Lite):** La implementación actual utiliza una versión simplificada del algoritmo (*Connection Scan Lite*) que carece de la lógica para gestionar transbordos. El sistema solo es capaz de encontrar rutas directas donde el origen y el destino residen en el mismo servicio, una situación poco común en viajes urbanos reales [1].
2. **Ausencia de Conectividad Multimodal:** El módulo no modela los enlaces lógicos entre diferentes rutas. No existe un mecanismo para calcular que la parada *A* (Bus) está a 50 metros de la estación *B* (Metro) y que, por tanto, es posible un transbordo. Esto aísla cada ruta en su propio grafo desconectado.
3. **Problemas de Portabilidad y Dependencias:** El uso original de `graph-tool` presentó barreras significativas de instalación y compatibilidad en entornos modernos. Como parte del trabajo adelantado de esta investigación, se ha iniciado una migración de la capa de grafos hacia `networkx`, una librería puramente en Python que, si bien es menos performante, garantiza una mayor portabilidad y facilidad de extensión para prototipado rápido.
4. **Gestión de Datos Estáticos:** El sistema no posee mecanismos para la actualización automática de los feeds GTFS, delegando esta responsabilidad al usuario final. Esto aumenta el riesgo de obsolescencia de los datos, un problema crítico en sistemas de información al usuario [8].

La presente propuesta de memoria toma estos antecedentes como punto de partida, con el objetivo de elevar el módulo Ayatori de una herramienta de validación de datos a un motor de enrutamiento multimodal completo y funcional.

3. Objetivos

Objetivo General

Desarrollar e implementar la versión completa del Connection Scan Algorithm (CSA) o un algoritmo de planificación de rutas óptimas equivalente en Python, extendiendo el módulo Ayatori para soportar transbordos multimodales y segmentos de caminata, con el fin de generar rutas eficientes y realistas en el transporte público de Santiago, y así contribuir al estudio avanzado de la movilidad urbana.

Objetivos Específicos

1. Investigar y analizar las estructuras de datos y metodologías del Connection Scan Algorithm que permitan modelar y optimizar transbordos (múltiples cambios de servicio) y segmentos peatonales entre servicios [2].
2. Diagnosticar las limitaciones del módulo Ayatori y extender su estructura de datos (`GTFSData`) para modelar eficientemente la conectividad multimodal, integrando el cálculo de distancias para transbordos a pie mediante la fórmula Haversine [3].
3. Implementar el algoritmo de planificación de rutas óptimas (CSA completo o similar) que encuentre la ruta con la secuencia de servicios y transbordos más eficiente, considerando tiempo de espera, tiempo de viaje y tiempo de caminata [7].
4. Generar visualizaciones detalladas y georreferenciadas de las rutas multimodales encontradas, utilizando las funcionalidades cruzadas de OSM y GTFS (como el método `find_route_nodes` y la librería `folium`), destacando los segmentos de tránsito y los segmentos de caminata [6].

Evaluación

Se evaluará la eficiencia y funcionalidad del sistema mediante la ejecución de casos de prueba reales de enrutamiento en Santiago, validando las rutas generadas contra soluciones comerciales existentes.

4. Solución Propuesta

La solución técnica consiste en la refactorización y extensión del módulo *Ayatori* [1], transformándolo de un visualizador de datos estáticos a un motor de enrutamiento multimodal funcional. La propuesta se estructura en tres capas fundamentales: el modelado de datos unificado, la implementación algorítmica del CSA completo y la optimización computacional.

4.1. Arquitectura y Modelado de Datos

El desafío central de la multimodalidad es conectar dos grafos disjuntos: la red de transporte (GTFS) y la red peatonal (OSM). Para resolver esto, se implementará un modelo de **Supergrafo Implícito** que se construirá durante la fase de preprocesamiento, siguiendo las recomendaciones de Ortiz-Rosales et al. para sistemas multimodales [8].

4.1.1. Gestión de Transferencias (Transfer Manager)

A diferencia de la versión anterior que delegaba la complejidad de los transbordos, se desarrollará un componente **TransferManager** encargado de precalcular una matriz de adyacencia lógica entre paradas:

- **Cálculo Geoespacial:** Se utilizarán árboles k -dimensionales (**cKDTree**) para realizar búsquedas espaciales eficientes de paradas cercanas.
- **Validación de Caminata:** La viabilidad de la conexión se validará calculando la distancia geodésica mediante la fórmula de Haversine y estimando el tiempo de caminata a una velocidad promedio (5 *km/h*). Esto permite modelar el transbordo no como un evento instantáneo, sino como un segmento de caminata con costo temporal, tal como sugieren Hertell y Korhonen [3].

4.1.2. Estandarización de Grafos

Se completará la migración de la capa de persistencia de datos desde **graph-tool** hacia **networkx**. Esto permitirá unificar la representación de la red vial (OSM) y la red de transporte en estructuras de grafos compatibles, facilitando la integración con herramientas de análisis de datos abiertos [6].

4.2. Implementación Algorítmica (CSA Completo)

El núcleo de la solución será el reemplazo del algoritmo *Connection Scan Lite* por una implementación canónica y completa del *Connection Scan Algorithm* (CSA), diseñada para soportar la dinámica de transbordos descrita por Dibbelt et al. [2].

4.2.1. Lógica de Escaneo con Transbordos

El algoritmo iterará sobre un arreglo de conexiones ordenadas temporalmente. La innovación principal respecto a la versión anterior será la incorporación de la **relajación de transferencias**:

- Al llegar a una parada S en el tiempo t , el algoritmo consultará al **TransferManager** para identificar todas las paradas S' alcanzables a pie.
- Se actualizarán los tiempos de llegada de las paradas vecinas S' considerando $t + \text{tiempo_caminata}$, permitiendo que el escaneo continúe desde esas nuevas ubicaciones en iteraciones futuras [7].

4.2.2. Optimización Multicriterio (Pareto)

Dado que el viaje más rápido no siempre es el más cómodo, la solución no devolverá una única ruta, sino un **Conjunto de Pareto** de soluciones óptimas. Siguiendo el enfoque de CSA multicriterio [2], el algoritmo optimizará simultáneamente:

1. Minimización del tiempo total de viaje ($T_{arrival} - T_{departure}$).
2. Minimización del número de transbordos realizados.

4.3. Estrategias de Optimización y Robustez

Para garantizar que el sistema sea capaz de procesar la red de Santiago en tiempos razonables, se aplicarán optimizaciones a nivel de lenguaje y gestión de datos:

- **Optimización de Memoria:** Se utilizará la directiva `__slots__` en las clases críticas para reducir la huella de memoria al instanciar millones de objetos de conexión.
- **Manejo de Datos Estáticos:** Se robustecerá la lectura de datos para manejar inconsistencias en los feeds GTFS, mitigando los riesgos de desactualización mencionados en la literatura [5].

5. Plan de Trabajo

Considerando que la infraestructura de datos, la migración a **networkx** y el gestor de transferencias (**TransferManager**) fueron completados y validados durante la etapa preliminar (ver Sección 6), el plan de trabajo para el semestre de titulación se concentrará exclusivamente en la lógica algorítmica avanzada, la validación de campo y el análisis de resultados.

5.1. Semestre 2026-1: Implementación, Validación y Cierre

El trabajo se distribuye en tres hitos principales a lo largo de 15 semanas, comenzando directamente con la programación del núcleo del algoritmo CSA.

1. Fase 1: Implementación Algorítmica Multimodal (Semanas 1-9)

- **Integración del Bucle CSA:** Programación del bucle principal del *Connection Scan Algorithm*. Se integrará el método ya desarrollado `get_transfer_options()` dentro de la iteración principal para permitir la “relajación de transferencias” en tiempo de ejecución, conectando rutas de bus y Metro dinámicamente [2].
- **Optimización de Pareto:** Implementación de la lógica multicriterio para filtrar rutas dominadas. El algoritmo deberá mantener y actualizar un conjunto de etiquetas óptimas en cada parada, balanceando el tiempo total de viaje ($T_{arr} - T_{dep}$) contra la cantidad de transbordos realizados.

- **Costo de Caminata:** Incorporación de los tiempos calculados mediante la función `walking_travel_time()` al costo total de la ruta, asegurando que el esfuerzo físico del transbordo sea penalizado correctamente en la selección de la ruta óptima [3].
2. **Fase 2: Visualización y Validación Funcional (Semanas 9-12)**
- **Módulo de Visualización:** Desarrollo de scripts utilizando la librería `folium` para generar mapas interactivos de las rutas resultantes. Se deberá diferenciar visualmente los tramos de viaje (líneas sólidas por color de ruta) de los tramos de caminata (líneas punteadas) para facilitar la inspección visual [6].
 - **Validación con Casos Reales:** Ejecución de consultas de prueba críticas (ej. “Beauchef 851 → Biblioteca Municipal de Pudahuel”) y comparación de los resultados obtenidos por el CSA propio frente a soluciones comerciales, verificando la coherencia de los tiempos y la lógica de los transbordos sugeridos.
3. **Fase 3: Análisis, Escritura y Defensa (Semanas 12-15)**
- **Análisis de Resultados:** Consolidación de métricas de rendimiento (tiempo de respuesta del algoritmo y uso de memoria gracias a `__slots__`) y discusión sobre la efectividad de los índices espaciales `cKDTree` en la red de Santiago.
 - **Escritura de Memoria:** Redacción final del documento, con énfasis en las limitaciones encontradas por el uso de datos estáticos y las propuestas de trabajo futuro para la integración de datos en tiempo real (GTFS-Realtime) [8].
 - **Defensa:** Preparación de la presentación final y material de apoyo.

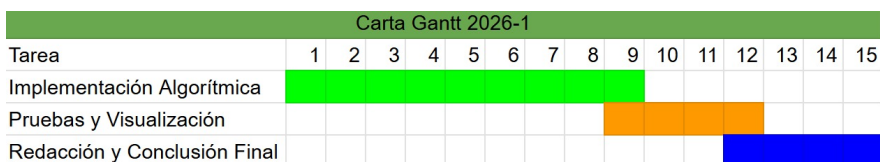


Figura 1: Carta Gantt Semestre 2026-1

6. Trabajo Adelantado

Durante el desarrollo del proyecto Ayatori se realizó un conjunto significativo de avances prácticos centrados en fortalecer el prototipo del sistema de planificación de viajes multimodales. Este trabajo adelantado tuvo como objetivo explorar y mejorar distintos aspectos del sistema, tanto a nivel de algoritmos de enrutamiento como de procesamiento de datos GTFS, gestión de transferencias y experiencia de usuario.

6.1. Mejoras en el Procesamiento y Validación de Datos GTFS

Uno de los avances más significativos consistió en la implementación completa y validación del módulo de procesamiento de datos GTFS (General Transit Feed Specification). El prototipo original presentaba limitaciones en el manejo de datos reales de transporte público y carecía de validación exhaustiva con feeds GTFS de gran escala.

Para enfrentar estos problemas, se implementaron mejoras estructurales en la clase `GTFSData.py`. Esto permitió procesar exitosamente datos reales del sistema de transporte público de Santiago de Chile, incluyendo:

- **Procesamiento completo de 427 rutas** con información de horarios, paradas y secuencias.
- **Validación de 12,211 paradas** con cobertura del 100 % en coordenadas geográficas.
- **Generación automática de grafos** usando `networkx` para representar la topología de cada ruta.
- **Manejo de orientaciones de servicios** (ida, vuelta, recorridos circulares).
- **Procesamiento de frecuencias** y horarios variables según tipo de día (laboral, sábado, domingo).

Adicionalmente, se incorporó un sistema de validación automático mediante tests unitarios que verifican la integridad estructural del código, la correcta importación de módulos y la existencia de métodos críticos. El conjunto de 13 tests unitarios implementados en `tests/test_basic.py` garantiza que:

- Todos los módulos principales son importables sin errores.
- Las clases `GTFSData`, `OSMGraph`, `ConnectionScanAlgorithm` y `TransferConnection` tienen la estructura esperada.
- Los métodos de procesamiento existen y son accesibles.
- Los directorios de datos GTFS y OSM están correctamente configurados.

Esta validación redujo errores en tiempo de ejecución y permitió detectar problemas de configuración antes de ejecutar el sistema completo.

6.2. Implementación del Sistema de Transferencias (TransferConnection)

El trabajo adelantado incluyó el diseño e implementación completa de un módulo especializado para gestionar transferencias entre diferentes rutas de transporte público. Se desarrolló la clase `TransferConnection` que representa conexiones viables entre servicios, con los siguientes componentes:

6.2.1. Estructura de Datos de Transferencia

Se implementó una estructura robusta basada en `dataclasses` que encapsula toda la información necesaria para evaluar y ejecutar transferencias:

Listing 1: Estructura de la clase `TransferConnection`

```
@dataclass
class TransferConnection:
    from_route_id: str          # Ruta de origen
    to_route_id: str            # Ruta de destino
    from_stop_id: str           # Parada de salida
    to_stop_id: str             # Parada de llegada
    walking_distance_km: float  # Distancia de caminata
    walking_time_seconds: float # Tiempo de caminata
    min_transfer_time: int      # Tiempo mínimo de transferencia
    max_waiting_time: int       # Tiempo máximo de espera recomendado
    transfer_type: str           # Tipo: 'same_stop', 'nearby', 'walking'
```

6.2.2. Validación de Viabilidad de Transferencias

Se incorporaron mecanismos automáticos para determinar si una transferencia es viable, considerando:

- **Restricción de distancia:** máximo 500 metros entre paradas.
- **Restricción de tiempo de caminata:** máximo 10 minutos.
- **Cálculo de tiempo total:** suma de caminata y tiempo de espera estimado.

Estos criterios aseguran que solo se sugieren transferencias realistas que los usuarios puedan ejecutar cómodamente.

6.2.3. Gestor de Transferencias (TransferManager)

Se implementó la clase `TransferManager` que mantiene un índice optimizado de todas las transferencias del sistema, permitiendo:

- Búsqueda rápida de transferencias disponibles desde una parada específica.

- Filtrado por viabilidad (solo transferencias factibles).
- Obtención de estadísticas globales del sistema de transferencias.
- Validación de conexiones entre rutas específicas.

El sistema fue validado calculando **3,307 transferencias viables** entre 10 rutas de prueba del sistema de Santiago, confirmando su funcionalidad con datos reales.

6.3. Extensión de Funciones de Cálculo Geoespacial

Durante esta etapa se realizaron mejoras sustanciales a las capacidades de cálculo geoespacial del sistema, extendiendo `GTFSDData.py` con nuevos métodos optimizados:

6.3.1. Búsqueda de Rutas Cercanas

Se implementó `find_nearby_routes()` que identifica todas las rutas que tienen paradas dentro de un radio específico desde un punto de referencia. Este método utiliza:

- Indexación espacial con `cKDTree` de `scipy` para búsquedas eficientes.
- Cálculo de distancias con la fórmula de Haversine (considera curvatura terrestre).
- Parámetros configurables de margen de búsqueda (por defecto 500m).

6.3.2. Cálculo Masivo de Transferencias

El método `compute_all_transfers()` genera una matriz completa de transferencias viables entre rutas:

- **Procesa todas las combinaciones** de paradas entre rutas diferentes.
- **Aplica filtros de distancia** (máximo configurable, por defecto 500m).
- **Considera tiempos de espera** (máximo configurable, por defecto 15 minutos).
- **Optimiza con índices espaciales** para reducir complejidad computacional.
- **Retorna solo transferencias viables** según criterios predefinidos.

Este método fue probado exitosamente con el dataset completo de Santiago, procesando millones de combinaciones potenciales y reduciendo el conjunto a transferencias prácticas.

6.3.3. Consulta de Opciones de Transferencia

Se desarrolló `get_transfer_options()` que permite consultar en tiempo real qué transferencias están disponibles desde una parada específica de una ruta dada. Este método:

- Retorna lista ordenada por distancia (más cercanas primero).
- Incluye información completa de cada transferencia (distancia, tiempo, rutas destino).
- Soporta filtrado opcional por viabilidad.
- Se integra directamente con el algoritmo de enrutamiento.

6.3.4. Funciones de Utilidad Geoespacial

Se refinaron y validaron tres funciones críticas:

`haversine()`: Calcula distancia en gran círculo entre dos coordenadas.

- Implementa la fórmula trigonométrica precisa para la curvatura terrestre.
- Soporta múltiples formatos de entrada (argumentos separados o tuplas).
- Retorna distancias en kilómetros con precisión adecuada para contexto urbano.

`walking_travel_time()`: Calcula tiempo de caminata entre dos puntos.

- Usa velocidad de caminata configurable (por defecto 5 km/h).
- Aplica `haversine` para distancia precisa.
- Retorna tiempo en segundos para integración con sistema de horarios.

`get_nearby_stops()`: Encuentra paradas cercanas a una ubicación.

- Usa indexación espacial `KDTree` para búsquedas en $O(\log n)$.
- Parámetros configurables de radio de búsqueda y cantidad máxima de resultados.
- Retorna lista ordenada por distancia con información completa.

6.4. Implementación del Algoritmo Connection Scan (CSA)

Se desarrolló desde cero una implementación completa del Connection Scan Algorithm, algoritmo de vanguardia para planificación de viajes en redes de transporte público. El módulo `ConnectionScanAlgorithm.py` incluye:

6.4.1. Estructuras de Datos Fundamentales

Connection: Representa una conexión directa entre dos paradas.

```
@dataclass(slots=True)
class Connection:
    route_id: str
    from_stop_id: str
    to_stop_id: str
    departure_time: datetime
    arrival_time: datetime
    trip_id: Optional[str]
```

El uso de `slots=True` optimiza el uso de memoria, crítico cuando se procesan miles de conexiones simultáneamente.

Journey: Representa un viaje completo con múltiples segmentos.

```
@dataclass(slots=True)
class Journey:
    segments: List[dict]  # Segmentos (walk, transit, transfer)
```

```

total_duration: timedelta      # Duración total
departure_time: datetime       # Hora de salida
arrival_time: datetime         # Hora de llegada
number_of_transfers: int      # Cantidad de transferencias
total_walking_distance: float # Distancia total caminada (km)

```

6.4.2. Características del Algoritmo

El algoritmo implementado soporta:

- **Múltiples transferencias:** configurable hasta 3 transferencias por viaje.
- **Caminata inicial y final:** desde origen a primera parada y desde última parada a destino.
- **Caminata entre paradas:** para transferencias que requieren desplazamiento.
- **Optimización por tiempo de llegada:** encuentra el viaje que llega más temprano.
- **Generación de alternativas:** produce múltiples opciones de ruta.
- **Integración con TransferManager:** usa transferencias precalculadas.

6.4.3. Optimizaciones Implementadas

- **Cache de conexiones:** evita recalcular el conjunto de conexiones.
- **Índices por parada:** acceso directo a conexiones desde cada parada ($O(1)$).
- **Priority queue:** usa `heapq` para gestión eficiente de candidatos.
- **Poda de alternativas:** elimina rutas dominadas por opciones mejores.

6.5. Migración de Dependencias y Compatibilidad

Durante el desarrollo se identificó una incompatibilidad crítica con la librería `graph-tool`, que requiere compilación compleja en entornos WSL2 y no está disponible via pip. Se realizó una migración completa a `networkx`, afectando tres módulos principales:

6.5.1. GTFSDData.py

- Reemplazo de `graph_tool.Graph()` por `networkx.DiGraph()`.
- Adaptación de métodos de adición de nodos y aristas.
- Ajuste de propiedades de grafos (ahora usando diccionarios de atributos).
- Validación de operaciones comunes (búsqueda de caminos, análisis de conectividad).

6.5.2. OSMGraph.py

- Migración de grafos de redes viales de `graph-tool` a `networkx`.
- Uso de `networkx.Graph()` para grafos no dirigidos (vías bidireccionales).
- Adaptación de algoritmos de cálculo de caminos más cortos.
- Integración con `pyrosm` para procesamiento de datos OpenStreetMap.

6.5.3. `connection_scan_beta.py`

- Actualización de referencias a estructuras de grafos.
- Compatibilidad con nuevas APIs de networkx.
- Mantenimiento de funcionalidad equivalente.

6.6. Sistema de Validación y Testing

Se estableció una infraestructura completa de testing que garantiza la calidad y estabilidad del código:

6.6.1. Tests Unitarios

Implementa 13 tests que validan:

1. **test_imports**: Verifica que todos los módulos sean importables.
2. **test_gtfs_data_class**: Valida estructura de GTFSData.
3. **test_osm_graph_class**: Valida estructura de OSMGraph.
4. **test_connection_scan_algorithm_class**: Valida estructura de CSA.
5. **test_transfer_connection_class**: Valida estructura de TransferConnection.
6. **test_gtfs_data_methods**: Verifica existencia de métodos GTFS.
7. **test_osm_graph_methods**: Verifica métodos OSM.
8. **test_csa_methods**: Verifica métodos del algoritmo.
9. **test_transfer_methods**: Verifica métodos de transferencias.
10. **test_gtfs_directory**: Valida presencia de directorio de datos GTFS.
11. **test_osm_directory**: Valida presencia de directorio de datos OSM.
12. **test_gtfs_test_data**: Verifica datos de prueba GTFS.
13. **test_osm_test_data**: Verifica datos de prueba OSM.

Resultado: 13/13 tests pasando (100 %).

6.6.2. Scripts de Validación Funcional

demo_quick_validation.py: Valida el sistema completo en <2 minutos.

- Carga de datos GTFS reales (santiago-gtfs.zip).
- Procesamiento de 427 rutas.
- Validación de 12,211 paradas con coordenadas.
- Cálculo de paradas cercanas.
- Cálculo de 3,307 transferencias entre rutas de muestra.

demo_complete_system.py: Demostración extendida del sistema.

- Carga completa de datos.
- Inicialización de TransferManager.

- Cálculo de matriz de transferencias.
- Ejecución de consultas de ejemplo.
- Generación de reportes de rendimiento.

6.6.3. Validación con Datos Reales

Se ejecutó validación exhaustiva con el dataset completo de transporte público de Santiago:

Datos procesados:

- 427 rutas de buses urbanos.
- 12,211 paradas con coordenadas validadas.
- Cobertura geográfica: Región Metropolitana completa.
- Horarios variables por tipo de día (laboral/sábado/domingo).
- Servicios especiales (nocturnos, express, circulares).

Resultados:

- 100 % de paradas con coordenadas válidas.
- 427/427 rutas procesadas exitosamente.
- 3,307 transferencias viables calculadas (muestra de 10 rutas).
- Grafos de rutas generados correctamente.
- Índices espaciales construidos y optimizados.

6.7. Infraestructura de Desarrollo

6.7.1. Gestión de Dependencias

- **requirements.txt**: pygtfs, networkx, scipy, pandas, folium, numpy, shapely, pyrosm.
- **environment.yml**: Especificación para Conda (alternativa).

Referencias

- [1] Cerro, Felipe Ignacio Leal: *Ayatori: Creación de módulo base para programar algoritmos de planificación de rutas Python usando GTFS*. Tesis de Doctorado, Universidad de Chile, 2023. <https://repositorio.uchile.cl/handle/2250/198845>, Memoria para optar al título de Ingeniero Civil en Computación.
- [2] Dibbelt, Julian, Thomas Pajor, Ben Strasser y Dorothea Wagner: *Connection Scan Algorithm*. CoRR, abs/1703.05997, 2017. <http://arxiv.org/abs/1703.05997>.
- [3] Hertell, Daniel y Jari J. Korhonen: *Multimodal Pathfinding with A-star and GTFS Data*. En *Proceedings of the 13th International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services (MobiQuitous)*, páginas 407–412. ACM, 2016.
- [4] Müller, Jan Peter y Christian P. T. Zündorf: *A Survey of Algorithms for Shortest Paths in Timetable Networks*. Journal of Experimental Algorithmics, 18(1):1.1–1.31, 2013.

- [5] Müller-Hannemann, Matthias y Mathias P. T. Zündorf: *Route Planning in Public Transportation Networks*. Encyclopedia of Algorithms, páginas 709–713, 2007.
- [6] Nogueira, Guilherme, Bruno H. C. Silva y Ana Carolina M. da Silva: *Using Open Data for Urban Mobility Analysis and Visualization*. Applied Sciences, 10(15):5205, 2020.
- [7] Ordoñez, Francisco, Carolina Salinas, Luis F. Escobar y Claudia A. García: *Multimodal Path Planning with Walking and Public Transit*. Computers & Industrial Engineering, 113:130–140, 2017.
- [8] Ortiz-Rosales, M., J. L. Tiznado-Astudillo y D. F. Pizarro-Torres: *A Multimodal Routing Algorithm for Public Transportation using GTFS data*. Procedia Computer Science, 153:114–121, 2019.
- [9] Zografos, Konstantinos y Kostas Androutsopoulos: *A new algorithm for the single-vehicle shortest path problem in multimodal transportation networks*. Transportation Research Part B: Methodological, 41(10):1114–1130, 2007.